

Introduction

Motivation

- En traitement d'image (classification, segmentation, reconnaissance) les réseaux de neurones avec des couches entièrement connectées permettent d'avoir des performances acceptables.
- Cependant, ces architectures présentent quelques limites :
  - Traiter les pixels de l'image tout en négligeant la structure spatiale de l'image
  - Croissance exponentielle du nombre de connexions avec la taille de l'image
  - Extraction manuelle ou à l'aide d'un algorithme des features : apprentissage supervisé
    - *(sinon considérer tout pixel comme feature)*

Introduction

Révolution en 2012

- ILSVRC (ImageNet Large Scale Visual Recognition Challenge)
  - Il s'agit d'une compétition de vision par ordinateur qui se tient chaque année depuis 2010. Son but est de localiser et classifier correctement des objets et scènes dans des images naturelles.
- AlexNet
  - Nouvel algorithme de réseau de neurones convolutif (Convolutional Neural Network : CNN) qui a marqué un tournant significatif dans le domaine de Deep Learning :
    - *Architecture profonde (huit couches)*
    - *Utilisation de GPUs*
    - *Méthode de régularisation (dropout)*
    - *Performance exceptionnelle et explosion des records.*

Méthodologie

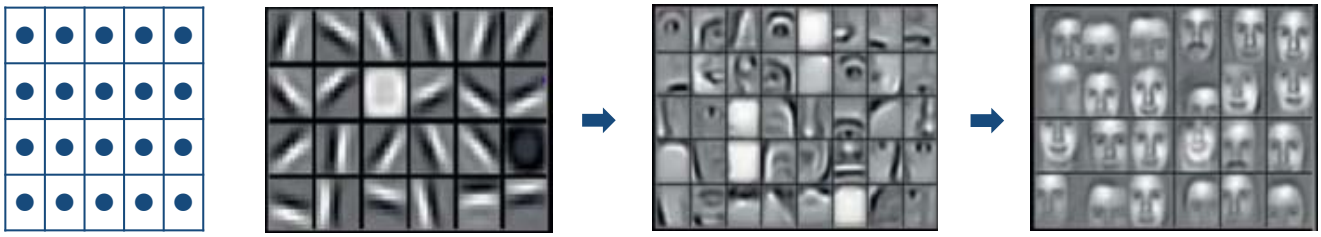
- Les CNNs ont une méthodologie similaire à celle des méthodes traditionnelles d'apprentissage supervisé : ils reçoivent des images en entrée, détectent les features, puis entraînent un classifieur dessus.
- Cependant, contrairement aux techniques d'apprentissage supervisé, les features sont apprises automatiquement :
  - Les CNNs réalisent eux-mêmes l'extraction et la description des features lors de la phase d'entraînement, l'erreur de classification est minimisée afin d'optimiser les paramètres du classifieur ET les features
- De plus, l'architecture spécifique du réseau permet d'extraire des features de différentes complexités : du simple au plus sophistiquée.
- L'extraction et la hiérarchisation automatiques des features, qui s'adaptent au problème donné, constituent une des forces des CNNs :
  - Plus besoin d'implémenter un algorithme d'extraction manuelle

Descripteurs

Descripteurs bas niveau (Low level features)

Descripteurs moyen niveau (Middle level features)

Descripteurs haut niveau (High level features)



Pixels

Contours (edges)

Parties d'objet (Object parts)

Objet (Object models)

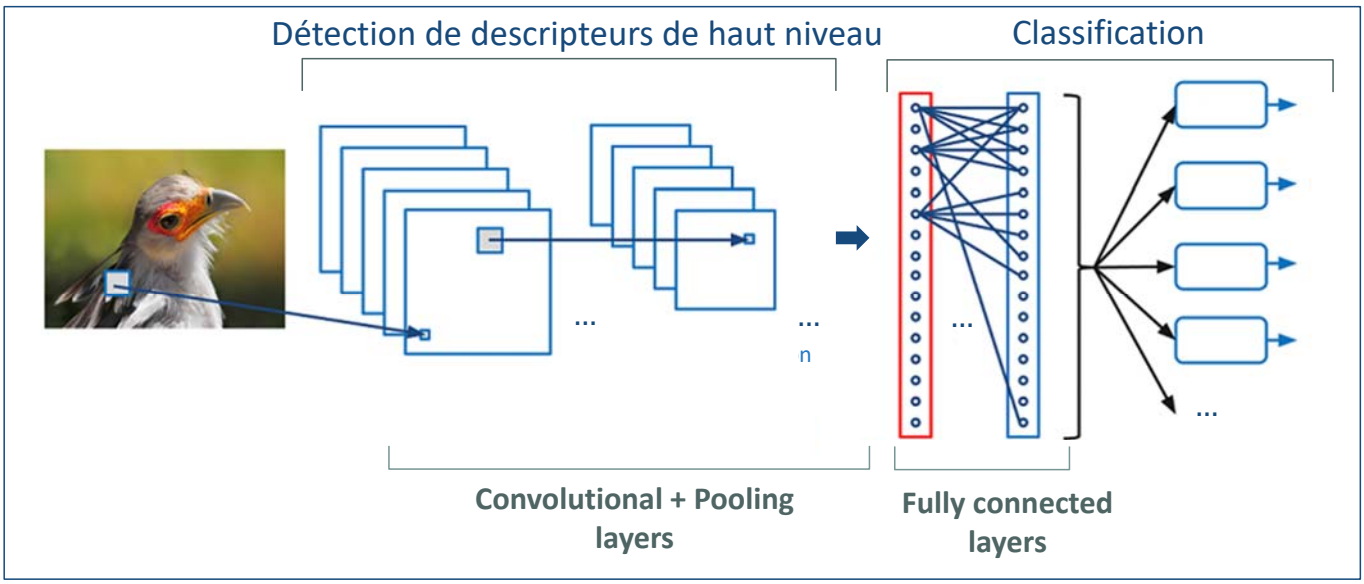
Lettre (Character)

Mot (Word)

Groupe de mot= phrase (Sentences)

Article (Story)

Architecture



Couches de CNN



CONVOLUTION

Appliquer des filtres pour extraire des descripteurs



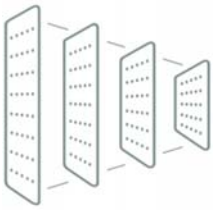
POOLING

Conserver les descripteurs importantes (réduction)



FLATTENING

Convertir les matrices de descripteurs en tableau 1D



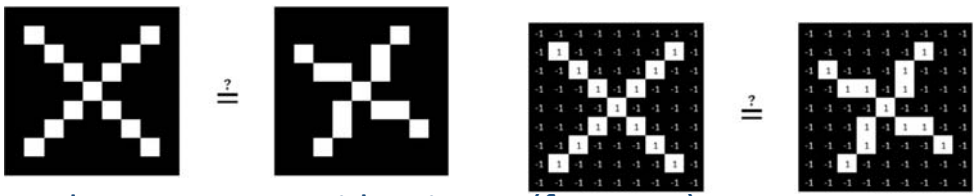
FULLY CONNECTED

Construire le modèle de régression/classification

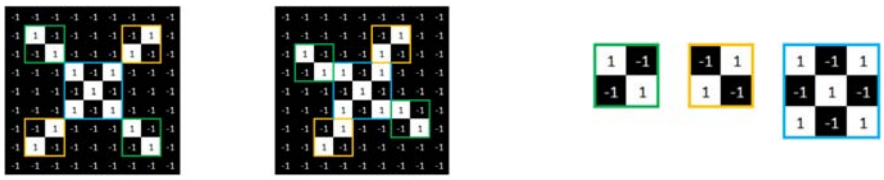
Couches de CNN

Convolution

- Deux images différentes



- Cependant, morceaux identiques (features)



- Considérons les features de taille 3×3 que nous souhaitons que la machine trouve



Couches de CNN

Convolution

- Étapes
  1. Multiplier (élément par élément) les valeurs des pixels de la feature avec les valeurs des pixels du morceau de l'image à trouver,
  2. Additionner ces résultats
  3. Diviser par le nombre de pixels
  4. Conclure (résultat égal à 1 → feature identifiée dans l'image)
- Exemple



- Le résultat de convolution est appelé image intermédiaire ou feature map
- Il est suivi d'une fonction d'activation telle que ReLU

Couches de CNN

Convolution

- **Paramètres**



`Depth (Volume)`

Nombre de filtres



`kernel_size`

Taille d'un filtre ou un noyau **K**



`Stride`

Détermine la façon dont le filtre est glissé sur l'entrée



`Padding`

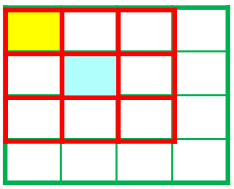
Détermine le nombre de pixels à considérer autour de la bordure

Couches de CNN

Convolution

- **Paramètres**
  - **padding="valid"**
    - *Pas de remplissage par des zéros*
    - *La taille sera réduite*
    - *Nombre de pixels de padding : **P = 0***
  - **padding="same"**
    - *Des zéros sont ajoutés au cadre extérieur de l'image (droite/Gauche/Bas/Haut) pour couvrir plus d'espace dans l'image*
    - *Taille d'entrée = Taille de Feature Map après convolution*
    - *Analyse plus précise des images*
    - *Nombre de pixels de padding : **P = (K-1)/2***

padding="valid"



padding="same"

0	0	0	0	0	0
0					0
0					0
0					0
0					0
0	0	0	0	0	0

Couches de CNN

Convolution : simple depth

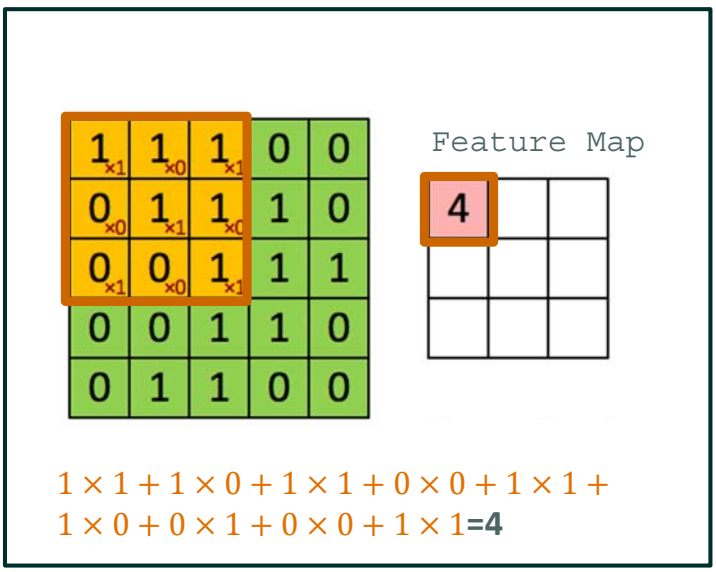
- **Input = 5×5**
- **kernel_size = 3×3**
- **Stride = 1**
- **Padding = "valid"**

Entrée

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filtre

1	0	1
0	1	0
1	0	1



Couches de CNN

Convolution : en volume

0	0	0	0	0	0	…
0	156	155	156	158	158	…
0	153	154	157	159	159	…
0	149	151	…	…	…	…
0	…	…	…	…	…	…
0	…	…	…	…	…	…
…	…	…	…	…	…	…

Input Channel 1

×

-1	-1	-1
0	1	0
0	1	1

Kernel 1 Channel 1

0	0	0	0	0	0	…
0	167	166	167	169	169	…
0	164	165	168	170	170	…
0	160	161	…	…	…	…
0	…	…	…	…	…	…
0	…	…	…	…	…	…
…	…	…	…	…	…	…

Input Channel 2

×

1	0	0
1	-1	-1
1	0	-1

Kernel 1 Channel 2

0	0	0	0	0	0	…
0	163	162	163	165	170	…
0	160	161	164	165	166	…
0	156	158	…	…	…	…
0	…	…	…	…	…	…
0	…	…	…	…	…	…
…	…	…	…	…	…	…

Input Channel 3

×

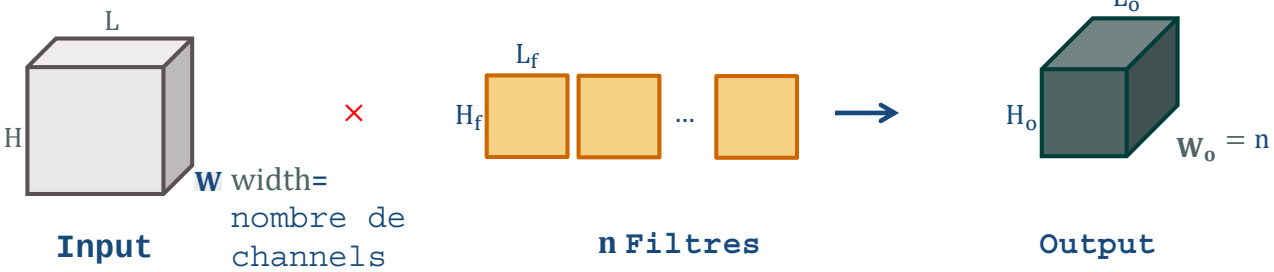
0	1	1
0	1	0
1	-1	1

Kernel 1 Channel 3

Couches de CNN

Convolution

- **Volume** $H_o * L_o * W_o$



$$H_o = \frac{H - H_f + 2*P}{stride} + 1 \qquad L_o = \frac{L - L_f + 2*P}{stride} + 1$$

- **Nombre de paramètres**

$$Nb_paramètres = (H_f \times L_f \times W + 1) \times n$$

Couches de CNN

Convolution

- **Conv2D (tf.keras.layers.Conv2D)**
  - *Structure par défaut*

    tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), activation=None, strides=(1, 1), padding='valid', input_shape=(None, None, 3))

  - *Exemple*

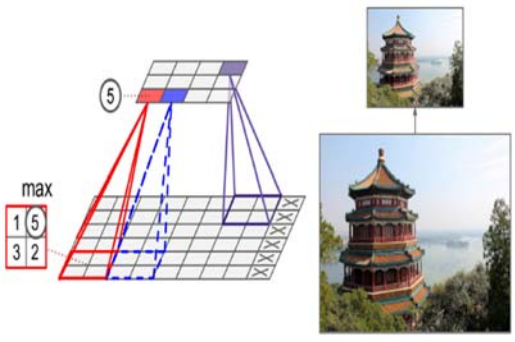
```
model = tf.keras.Sequential()

model.add(tf.keras.layers.Conv2D(20, (3, 3),
                strides=(1,1), padding="same", activation="relu",
          input_shape=(64,64,3)))
```


Couches de CNN

Pooling

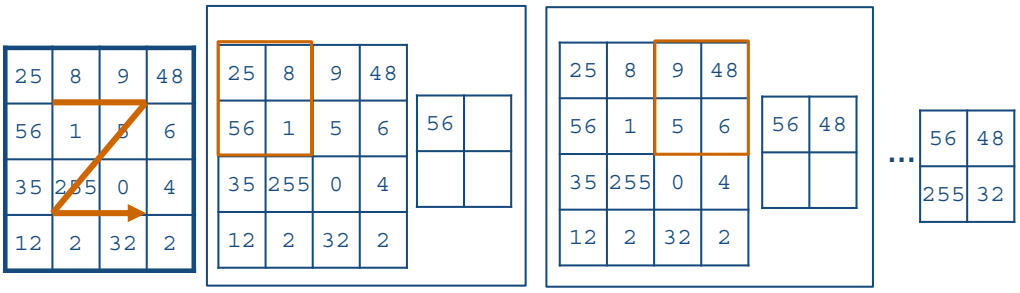
- C'est une technique qui consiste à réduire les dimensions des feature map en remplaçant un bloc par une seule valeur
- Il existe deux techniques:
  - **mean-pooling**
  - **max-pooling (la plus utilisée)**
    - *Elle prend la valeur maximale de chaque morceau de l'image*



Couches de CNN

Pooling

- **Exemple**
  - MaxPooling
  - pool_size= 2×2
  - padding="valid"
  - Stride=2



Couches de CNN

Pooling

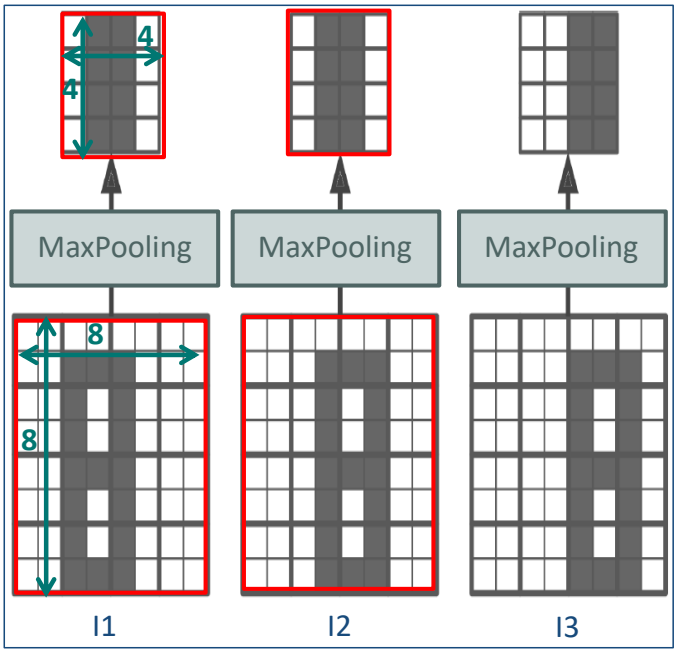
- **Intérêts**
  - Prévenir le sur apprentissage
  - Obtenir des données de longueur fixe
  - Invariance
    - *Fait référence à la capacité du modèle à maintenir certaines caractéristiques importantes malgré des variations dans la position spatiale*
    - *Reconnaître des objets même s'ils sont légèrement décalés ou positionnés différemment dans l'image*

Couches de CNN

Pooling

- **Intérêts**
  - **Exemple**
    - *MaxPooling*
    - *pool_size= 2×2*
    - *padding="valid"*
    - *Stride=2*

I1=I2

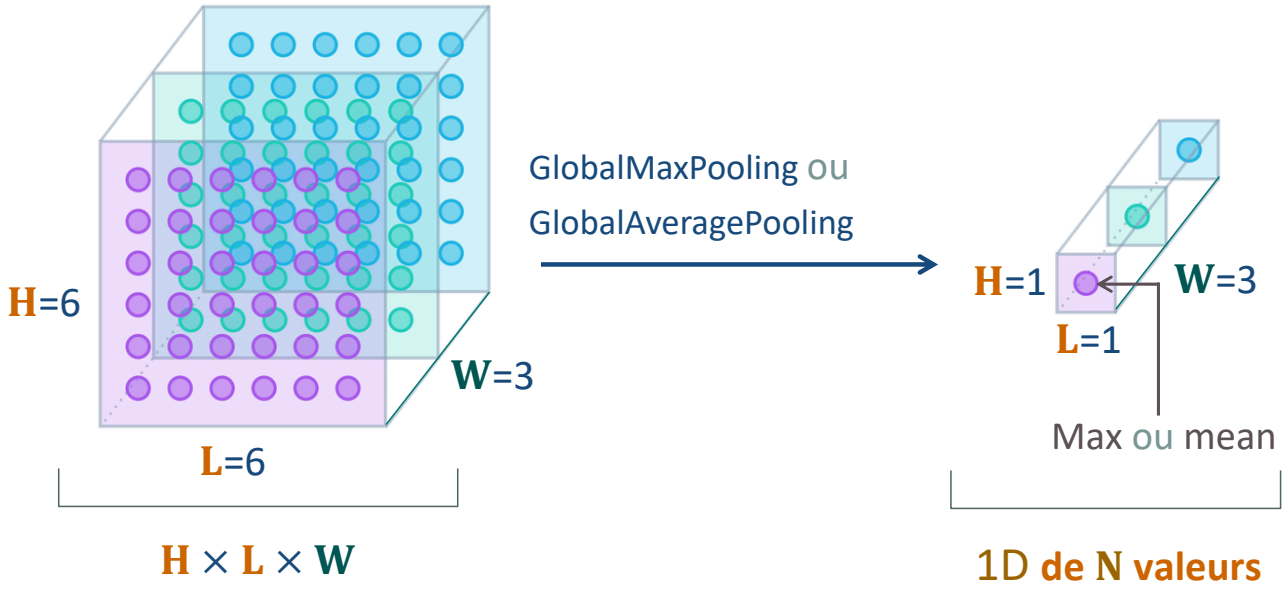


I1, I2 et I3:Mêmes images mais chacune est décalée de 2 pixels vers la droite

Couches de CNN

Pooling

- **GlobalMaxPooling/GlobalAveragePooling**



Couches de CNN

Pooling

- MaxPooling2D/ AveragePooling2D/ GlobalMaxPooling2D
  - (tf.keras.layers.MaxPooling2D/AveragePooling2D)
  - Structure par défaut

```
tf.keras.layers.MaxPooling2D(pool_size=(2, 2), strides=None, padding='valid')

tf.keras.layers.AveragePooling2D(pool_size=(2, 2), strides=None, padding='valid')
```

  - Exemples

```
model.add(tf.keras.layers.MaxPooling2D(pool_size=(4, 4),
        strides=(4,4), padding="valid"))

model.add(tf.keras.layers.AveragePooling2D(pool_size=(4, 4),
        strides=(4,4), padding="valid"))
```

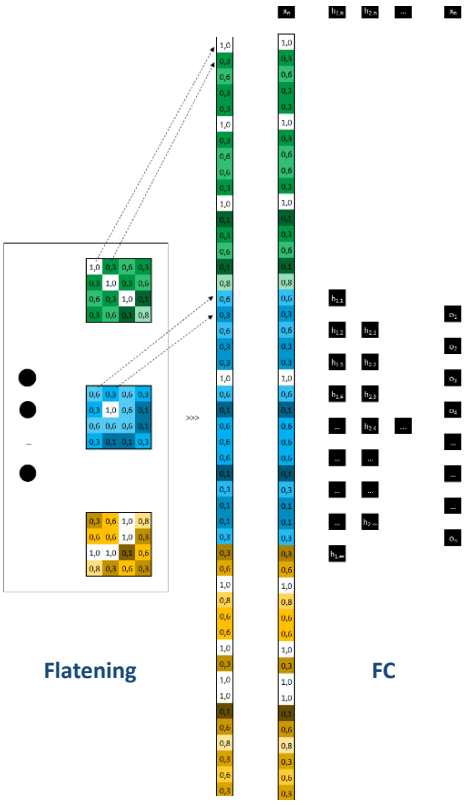

Couches de CNN

Flattening

- Il consiste tout simplement à prendre la totalité des valeurs de nos matrices précédemment calculées, et à les empiler, en vue de les exploiter dans la couche d'entrée d'un réseau de neurone

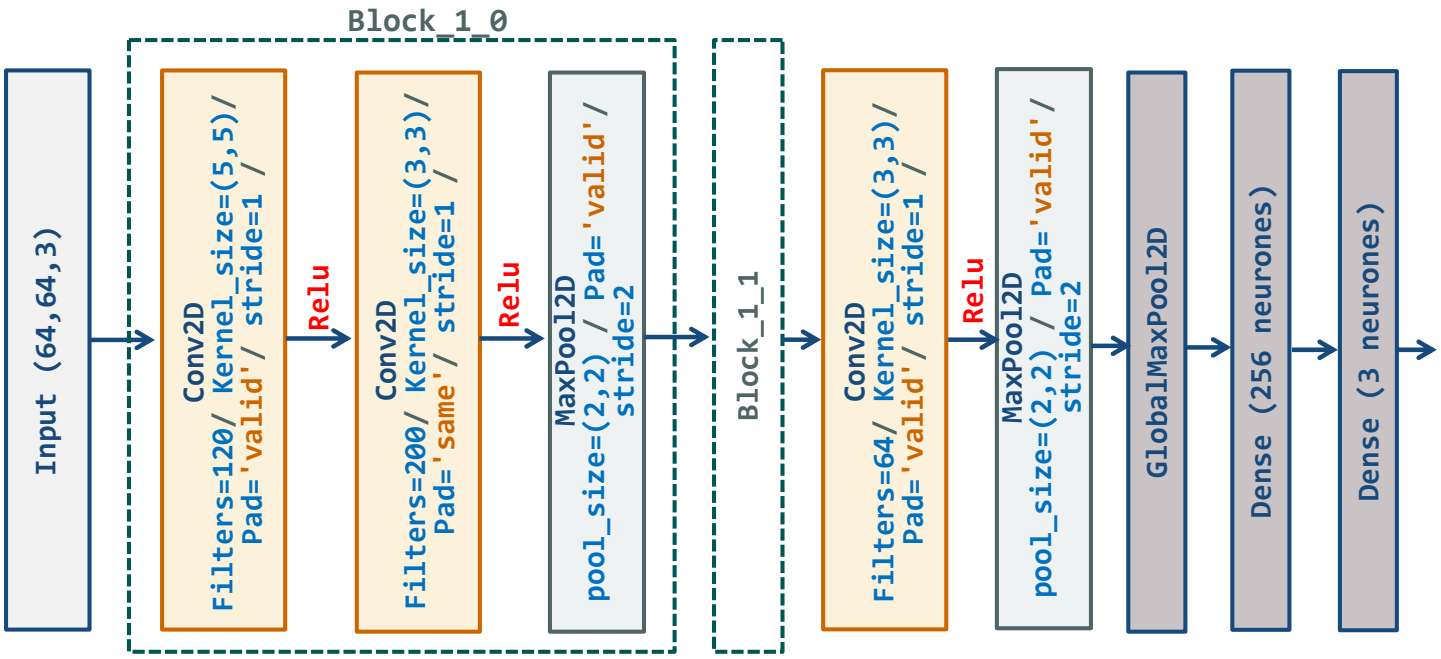
Réseau de neurones fully connected (FC)

- On utilise dans la couche d'entrée les valeurs calculées précédemment, on ajoute ensuite de 1 à une infinité de couches cachées à notre réseau. Enfin, on ajoute une couche de sortie avec le nombre de neurones adéquats



Couches de CNN

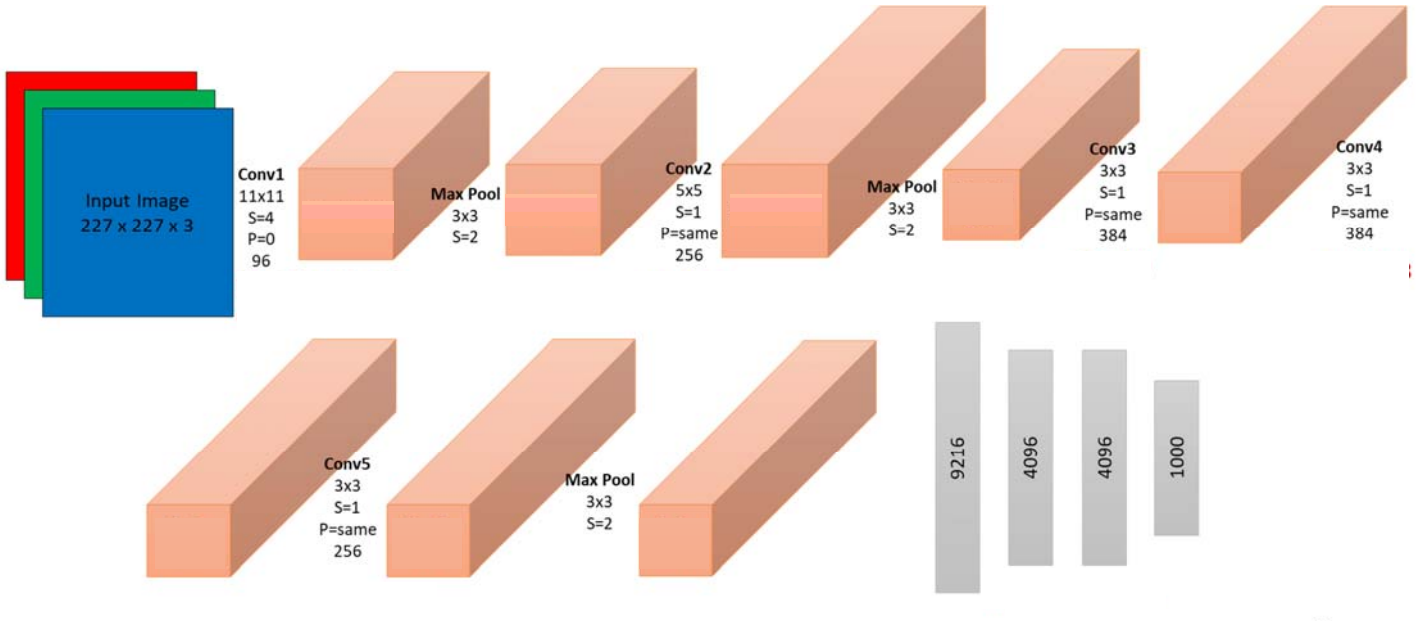
Exemple



Couches de CNN

Calcul des paramètres : AlexNet 2012

Total Parameters = 62.37 M
Conv Layer = 3.7M = 6%
FC = 58.6M = 94%

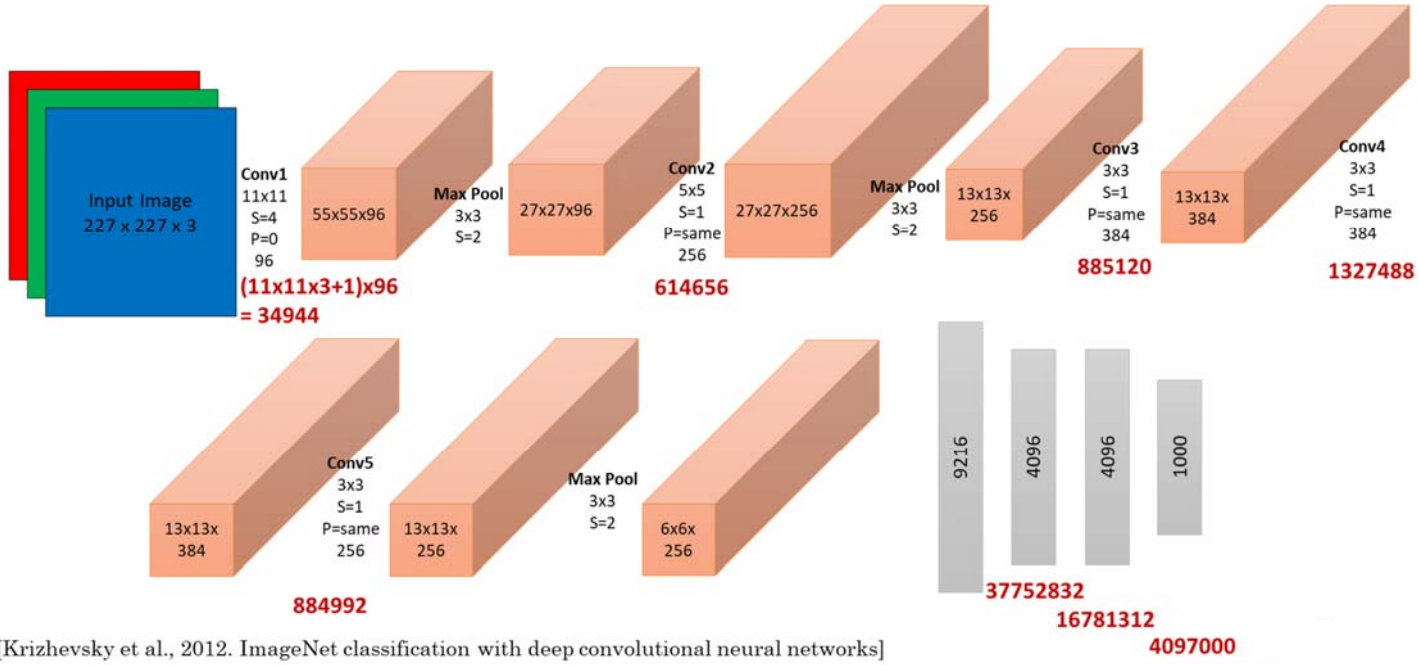


[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Couches de CNN

Calcul des paramètres : AlexNet 2012

Total Parameters = 62.37 M
Conv Layer = 3.7M = 6%
FC = 58.6M = 94%



[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Enregistrement

Checkpointing *.keras* (architecture+poids), *.weights.h5* (poids)

- **Par défaut**

```
tf.keras.callbacks.ModelCheckpoint(filepath, monitor='val_loss',
        save_best_only=True, save_weights_only=False, mode='auto')
```

- **monitor='val_loss'**, **'loss'**, **'val_accuracy'** ou **'accuracy'**
- **(save_best_only=True)** Enregistrer uniquement le meilleur modèle selon **monitor** (sinon chaque époque)
- **(save_weights_only=False)** Sauvegarder le modèle complet (architecture + poids)
- **mode='auto'** (selon monitor), **'max'** (accuracy) ou bien **'min'** (loss)

- **Exemple**

```
model = tf.keras.Sequential()
:
checkpoint = tf.keras.callbacks.ModelCheckpoint(filepath='../checkpoint.keras')
history=model.fit(..., callbacks=[checkpoint])
```

Utilisation

Modèle entier

```
model = tf.keras.models.load_model(filepath='../ checkpoint.keras')
                                    ⋮
```

Poids enregistrés

```
model  = tf.keras.Sequential()
                    ⋮
model.load_weights(filepath='../checkpoint.weights.h5')
```


Quiz

1. Quel est l'intérêt d'ajouter une couche de Pooling lors de la création d'un réseau de neurones convolutif ? (seule réponse)

- [x] Réduire la taille de `Feature maps`
- [ ] Extraire les descripteurs de l'image
- [ ] Construire un modèle
- [ ] Obtenir des données de longueur variable

2. Parmi les éléments suivants, lequel peut être l'entrée de la couche "Fully connected" (une seule réponse)

- [ ] Un vecteur `2D` de taille variable
- [ ] Un vecteur `1D` de taille variable
- [ ] Un vecteur `2D` de taille fixe
- [x] Un vecteur `1D` de taille fixe

3. Les réseaux de neurones convolutifs permettent d'extraire automatiquement des descripteurs de hauts niveaux à partir des images (Vrai/Faux)

- [x] Vrai
- [ ] Faux